



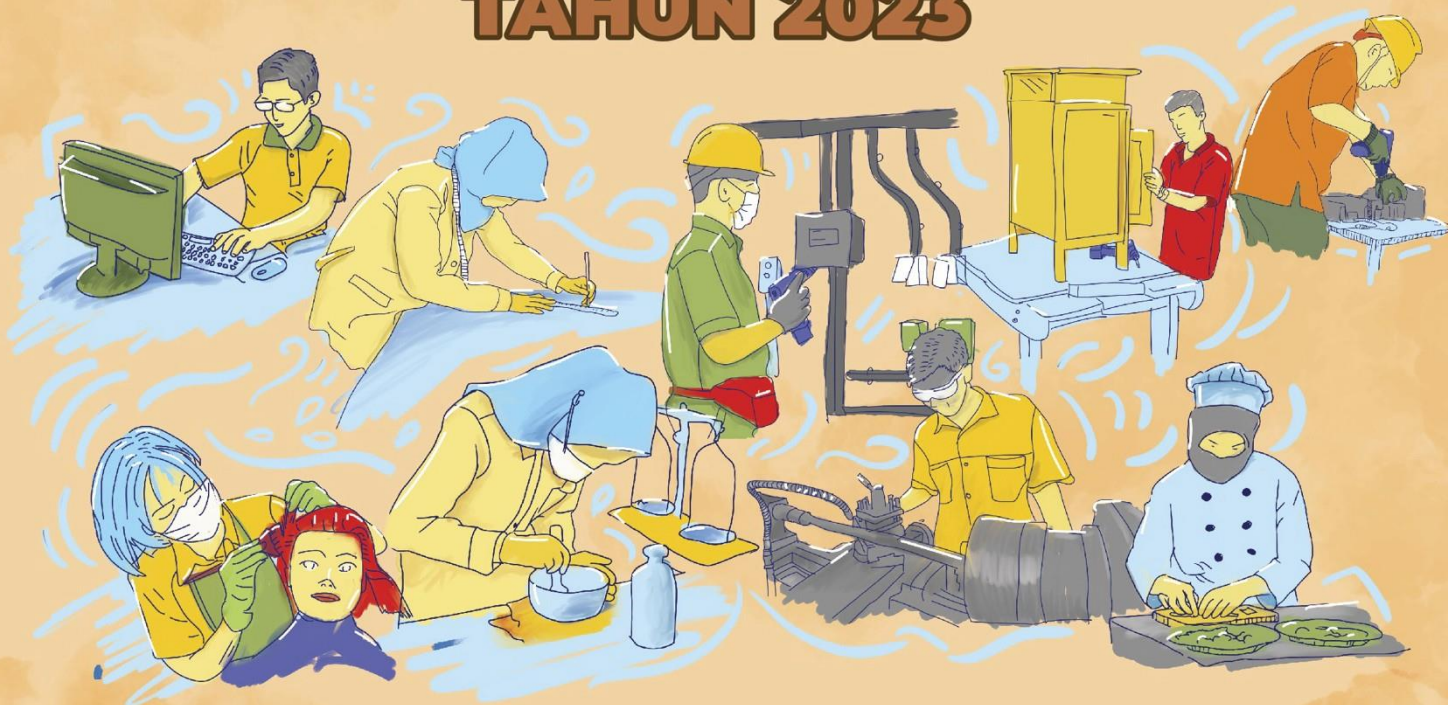
BALAI PENGEMBANGAN TALENTA INDONESIA
PUSAT PRESTASI NASIONAL
SEKRETARIAT JENDERAL
KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI

**MERDEKA
BELAJAR**



SOAL

LOMBA KOMPETENSI SISWA SMK TINGKAT NASIONAL TAHUN 2023



BIDANG LOMBA

Komputasi Awan
(Cloud Computing)

MERDEKA BERPRESTASI
Talenta Vokasi Menginspirasi

Description of project and tasks

This module is eight hours - **Day 1 - Project 1: Serverless Architecture.**

The goal of this project is to deploy an online concert ticket sales web application organized by Sound Wave Production promoter. You are required to deploy the front-end application and implement a serverless architecture on the backend side in accordance with the system design created by the development team. You must also apply a serverless architecture that is reliable, secure, high-performing, and cost-efficient.

Task

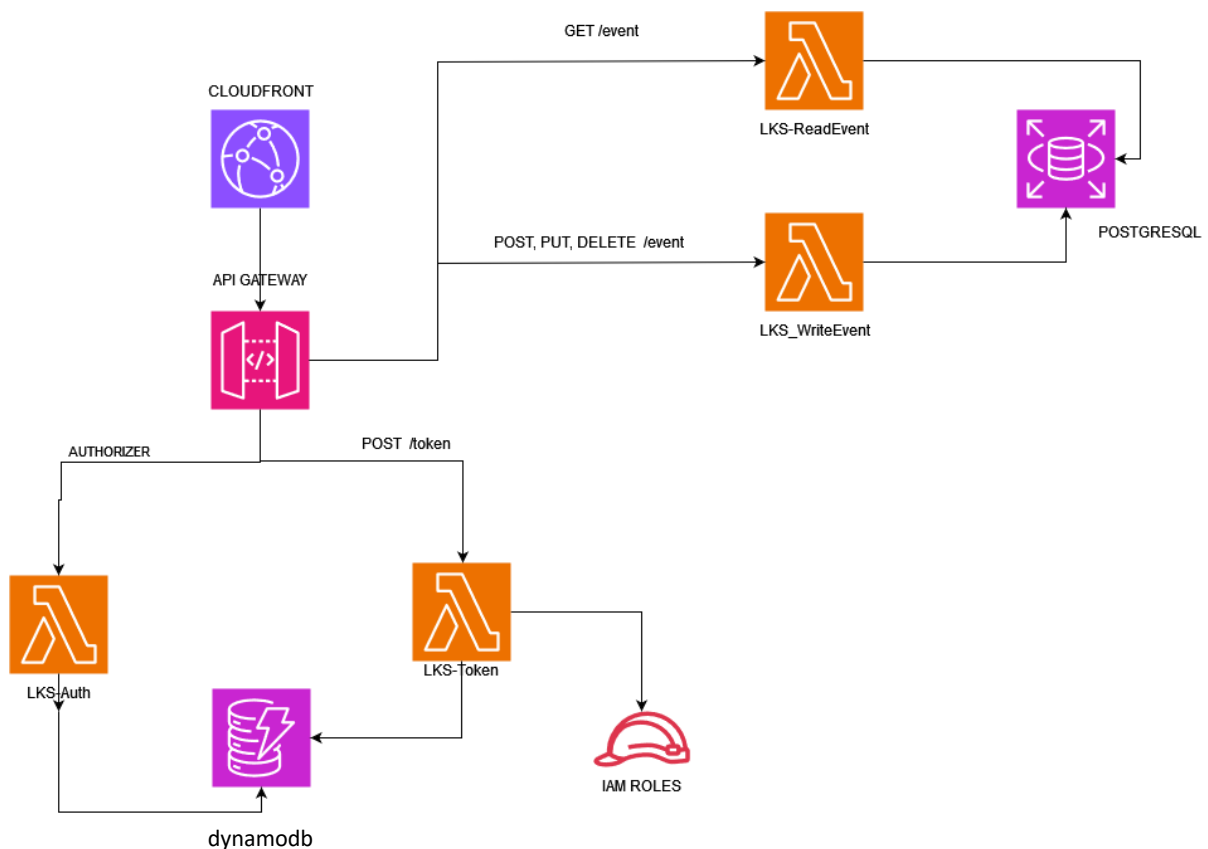
1. Read the documentation thoroughly (Outlined below).
2. Please read and understand the application architecture in the **Architecture section**.
3. Please carefully read the **technical details** section.
4. Please carefully read the **application details**.
5. Log in to the AWS console.
6. Set up your VPC configurations. The VPC configuration details are in the **Network Architecture - Service Details** section.
7. Set up relational database and non relational database. You can read more in **Database - Service Details** and **DynamoDB - Service Details** section.
8. Set up a security group. You can read security additional rules in **Security - Service Details** section.
9. Set up the API Endpoint and Websocket in **API Gateway - Service Details** section.
10. Prepare the IAM and SSM Parameter Store to provide access and configuration for the application.
11. Configure necessary application monitoring and metrics in CloudWatch.

Technical Details

1. The goal of this project is to deploy the serverless application that has been developed until it runs successfully.
2. All of your service must be place in region **us-east-1**.
3. Gather all the policies required by your service in a single role and name it '**lks-role**', so that it will be easier for the judges to evaluate the policies you have created.
4. All the necessary resource source code is available on GitHub at <https://github.com/betuah/lks-serverless>.
5. This project places a strong emphasis on high-level security to safeguard sensitive data and mitigate potential threats effectively.

6. Every service you have created must use the "lks-" naming format (e.g., "lks-allow-http," "lks-auth," "lks-apiGateway," etc.). The judges will only consider work that follows this recommended naming format.
7. Ensure that you label each AWS service you create, except for those that were automatically generated. Paying attention to these details will contribute to earning more points.
8. Remember to fill in each description and tag of the services.
9. Remove unnecessary service items, keep your work clear for easy assessment by the jury, and avoid potential point deductions.
10. The programming language utilized in this project is JavaScript, using the NodeJS version 16 runtime.

Architecture



The above example illustrates one possible architectural design for the Concert Ticket Application. This is not the final architecture that you may follow. This architecture shows the design system built by the application development team to make it easier to comprehend how the application operates. Please read the Application detail.

Application Details

In this project, you will deploy a web application for concert tickets that implements a serverless architecture on the backend. You are required to deploy backend applications until they are running and can be used properly.

The backend application has already been developed to supports a serverless architecture with lambda functions. All lambda functions needed for the backend application are already available. You may only need to write a few lines of your own lambda function code according to the architecture requirements.

The architecture diagram is provided in the Architecture section. The source code can be accessed at the following repository link: <https://github.com/betuah/lks-serverless>.

Service Details

Networking (VPC)

Although this is a serverless application, in this case, you still need to build network infrastructure for the database tier. You must create a VPC in the specified region with CIDR 172.32.0.0/16. You are required to create 2 public subnets and 2 private subnets, each in a different availability zone. Each subnet requires only 200 IP addresses. If you need to link a private subnet to another AWS service, choose the most cost-effective option.

Security

Security is the most important aspect to consider when building a serverless architecture. In this case, you are required to secure the API gateway from potential attacks such as DDoS attacks and SQL Injection. You are also required to secure the RDS database, ensure that the RDS can only be accessed by lambda functions that require a connection to the database and ensure that the database cannot be accessed publicly. In addition, you also need to consider the security of other services used in this project. Use AWS IAM to provide policies that meet your needs. Remember that granting excessive access will create security vulnerabilities in your architecture.

SSM Parameter Store

Every lambda function in this project that performs CRUD operations on AWS RDS will retrieve database configuration data from the SSM Parameter Store using the path `/lks/database/{{KEY}}`. The key must be the same as the name of the required database configuration. The following keys must be created in the parameter store:

Key	Type	Description
dbname	String	Database name
username	String	Database username
password	SecureString	Database password
endpoint	String	Database host endpoint

Relational Database

In this project, the backend application uses the RDS service as its primary database. Create a database serverless v2 with PostgreSQL as the engine type that supports this backend application. The database settings can be found in the connection.js file in the service folder at the provided source code. Setup username, password and initial database name that are required for backend and store to SSM Parameter store, please read SSM Parameter service details. The database must be configured to be highly available, scalable, and secure.

DynamoDB

A custom authorizer will be used in this serverless architecture to manage authorization security in API Gateway by evaluating the permitted token and deviceid data. In DynamoDB, create a table called "tokens" with the partition key "token" (S) and the sort key "deviceid" (S). Then, for RCU and WCU, configure the token table to utilize provisioned mode with auto scaling enabled. Set the tokens RCU min to 10 and max to 100 with an 70% utilization target. Also, set the tokens WCU min to 5 and max to 25 with an 70% utilization target.

Lambda Function

The detailed configuration for the lambda function can be found in the documentation of the source code repository at <https://github.com/betuah/lks-serverless>.

API Gateway

API Gateway is a critical component of this project. You must carefully create API endpoints that adhere to the architecture design of this project. Two types of API Gateway will be used in this project REST API. The following is the API Gateway configurations required for this project.

1. REST API

The REST API uses an edge-optimized endpoint type accessed from Cloud Front with a custom authorizer named **lks-auth**. The authorizer invokes a Lambda function that contains the auth.js file. The authorizer is configured to use the **Authorization** header and **Deviceid** as the identity source. The API Gateway will also accept file uploads in base64 format. The only media files that are allowed to be uploaded are **image/jpg** and **image/png**. Use production as a stage name and perform caching. The following is a table of REST API endpoints required:

Resource	Method	Authorizer	CORS	Service Type	Execution
/token	POST	AWS IAM	No	Lambda	lks-token
/event	GET	lks-auth	Yes	Lambda	lks-read-event
/event	POST,PUT,DELETE	lks-auth	Yes	Lambda	lks-write-event

You may test the API Endpoint by going to the API documentation at this [link](#).